

Meta-Learning with Heterogeneous Tasks

Zhaofeng Si¹, Shu Hu², Kaiyi Ji¹, Siwei Lyu¹

¹University at Buffalo, State University of New York

²Purdue University

zhaofeng@buffalo.edu, hu968@purdue.edu, kaiyiji@buffalo.edu, siweilyu@buffalo.edu

Abstract

Meta-learning is a general approach to equip machine learning models with the ability to handle few-shot scenarios when dealing with many tasks. Most existing meta-learning methods work based on the assumption that all tasks are of equal importance. However, real-world applications often present heterogeneous tasks characterized by varying difficulty levels, noise in training samples, or being distinctively different from most other tasks. In this paper, we introduce a novel meta-learning method designed to effectively manage such heterogeneous tasks by employing rank-based task-level learning objectives, *Heterogeneous Tasks Robust Meta-learning* (HeTRoM). HeTRoM is proficient in handling heterogeneous tasks, and it prevents easy tasks from overwhelming the meta-learner. The approach allows for an efficient iterative optimization algorithm based on bi-level optimization, which is then improved by integrating statistical guidance. Our experimental results demonstrate that our method provides flexibility, enabling users to adapt to diverse task settings and enhancing the meta-learner’s overall performance.

1 Introduction

Deep neural network models trained with supervised learning have made significant progress in computer vision, in many cases surpassing human abilities. Conventional supervised learning needs a large number of training samples, but in real-world situations such as image classification (Russakovsky et al. 2015), models must tackle multiple tasks that arise during the learning process, each with only a few samples available. This scenario is often referred to as *few-shot* learning. Meta-learning, or learning to learn, has been proposed as a solution to few-shot learning. In particular, optimization-based meta-learning methods introduce a meta-model (Finn, Abbeel, and Levine 2017; Huisman, Van Rijn, and Plaat 2021), whose parameters can be fine-tuned for individual tasks using a few gradient descent update steps. In this way, the meta-model can use the knowledge gained from previous training tasks to enable the learning model to adapt rapidly to new tasks during meta-testing.

Current meta-learning methods often implicitly assume that all tasks are more or less equal in importance. However, in real-world applications, tasks are often heterogeneous in the level of learning difficulty, data quality, and domain consistency. We use image retrieval as an example to demonstrate the situation, Figure 1. Specifically, we

can identify three types of tasks: (a) Harder tasks, which are tasks with ambiguous contexts (row 1: wolf vs. husky) are more challenging for the model; (b) Noisy tasks: tasks with corrupted labels (row 3, first sample mislabeled as “boxer” and the query mislabeled as “lion”) are more prone to errors; and (c) Outlier tasks that are considerably different from most training tasks (row 3 birds vs. row 1-2 airplanes). We can often differentiate these tasks using the task-level losses on actual multi-task meta-learning tasks, as shown in the bottom row of Figure 1. It shows a histogram that outlines the distribution of task-level losses by conducting fast adaptation using a MAML (Finn, Abbeel, and Levine 2017) trained meta-model on clean tasks from Mini-ImageNet, noisy (tasks derived from Mini-ImageNet but with labels randomly flipped), and outlier tasks with classes from Tiered-ImageNet that do not overlap with those in Mini-ImageNet tasks.

The heterogeneity introduces challenges for the meta-training step. Several works have concentrated on improving the effectiveness of meta-learning algorithms at the task level to address challenges posed by varying task difficulties (Collins, Mokhtari, and Shakkottai 2020; Nguyen, Do, and Carneiro 2023; Liu et al. 2023) and tasks with noise (Killamsetty et al. 2022; Liu et al. 2020). However, limited works studied the effects of outlier tasks (also known as Out-of-Distribution tasks in (Killamsetty et al. 2022)) in meta-learning and the method of handling multiple heterogeneous tasks concurrently. A substantial body of literature has addressed the development of robust meta-learning algorithms to mitigate the impact of corrupted labels (Wang, Hu, and Hu 2020; Ren et al. 2018b; Killamsetty et al. 2022; Lee et al. 2018; Liu et al. 2020). Nevertheless, these works predominantly concentrate on sample-level noise, with limited attention directed to examining robustness at task level.

In this paper, we introduce the *Heterogeneous Tasks Robust Meta-learning* (HeTRoM) method to address the problem posed by the heterogeneous tasks in meta-learning. The core component of HeTRoM is a new rank-based task-level loss that enables the dynamical selection of tasks in the meta-training step. Specifically, the HeTRoM loss controls the cut-off range in the ranked individual task losses, which reduces the influence of tasks whose losses are either too small (easy tasks) or too large (noisy tasks and outlier tasks). The hyper-parameters determining the range can be

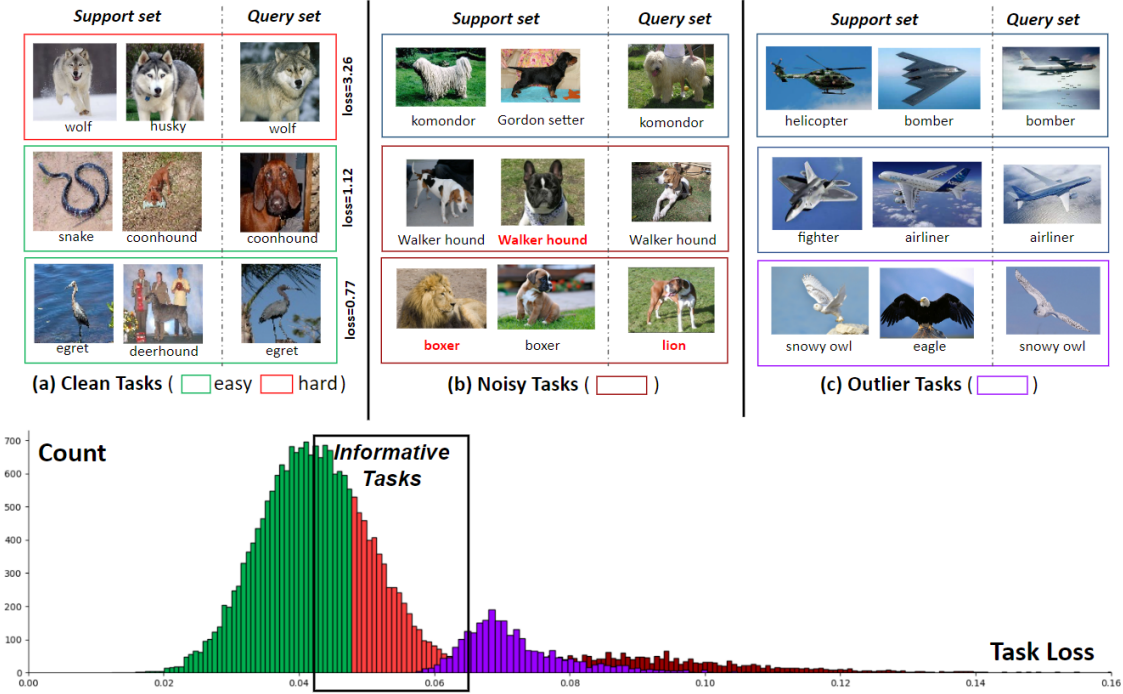


Figure 1: *Heterogeneous tasks in meta-learning for image retrieval on real-world datasets (Mini-ImageNet and Tiered-ImageNet). (Top) Three different types of tasks based on their difficulty. (Bottom) Histograms of losses can be used to differentiate three types of tasks. The displayed histogram is derived from real-world datasets, the details are provided in texts.*

obtained based on a log-normal model of the distribution of the individual task losses. We show that the HeTRoM objective can be formulated as a bi-level optimization problem and efficiently solved with a stochastic gradient-based algorithm to update the meta-model. Experimental evaluations on two benchmark datasets underscore the effectiveness and robustness of HeTRoM.

2 Related Works

As the most representative optimization-based approach, Model Agnostic Meta-learning (MAML) (Finn, Abbeel, and Levine 2017) learns an initialization such that a gradient descent procedure starting from the initial model can achieve fast adaptation. In the following years, a large number of works on various MAML variants have been proposed (Grant et al. 2018; Finn et al. 2019; Raghu et al. 2020).

Another group of meta-regularization approaches has been proposed for improving the bias for a regularized empirical risk minimization problem (Alquier, Pontil et al. 2017; Balcan, Khodak, and Talwalkar 2019; Zhou et al. 2019). In addition, there is a common embedding-based framework in few-shot learning (Bertinetto et al. 2018; Lee et al. 2019; Zhou, Wu, and Li 2018). The goal of this framework is to learn a shared embedding model that can be used for all tasks. Task-specific parameters are then learned for each task based on the embedded features.

There exists a body of literature dedicated to investigating the robustness of optimization-based meta-learning. Some of these studies concentrate on addressing noise at the

sample level within few-shot tasks. (Killamsetty et al. 2022; Malleem, Hasnat, and Nakib 2023; Sun et al. 2022). For example, Eigen-Reptile (Chen et al. 2022) is a method that derives the updating direction of the meta-model by calculating the main direction of fast adaptation, which mitigates the impact of corrupted data within tasks.

There are also several works focusing on task-level robustness (Yao et al. 2021; Collins, Mokhtari, and Shakkottai 2020; Killamsetty et al. 2022; Liang et al. 2022). Yao et al. 2021 propose to use an adaptive task scheduler (ATS) to decide the next tasks to be used while training to avoid the effect of tasks with corrupted labels. NestedMAML (Killamsetty et al. 2022) uses a weighted sum of instances or tasks in meta-training for handling Out-of-Distribution (OOD) tasks or noisy data, where the weights are updated through a nested bi-level optimization approach. While this re-weighting approach can reduce the effect of corrupted samples and tasks, it cannot eliminate them entirely. Liang et al. 2022 focus on improving the robustness of metric-based meta-learning, which is not within the scope of this paper, as we concentrate on optimization-based meta-learning.

3 Background

Meta-learning is commonly formulated as an optimization problem (Huisman, Van Rijn, and Plaat 2021). Let $\{\mathcal{T}_i\}_{i=1}^n$ be a collection of n tasks. Each task $\mathcal{T}_i$ has a specific loss function $\mathcal{L}(\phi, w_i; \xi)$ in sample ξ , where ϕ are the parameters of the shared embedding model for all tasks and w_i are the task-specific parameters. We aim to learn the optimal pa-

parameters ϕ that work well for all tasks, and each task uses the optimized ϕ to adapt its specific parameters w_i minimizing $\mathcal{L}(\phi, w_i; \xi)$. For instance, if the learning task is solved with a series of deep neural network models, w_i may correspond to the parameters of the final layer within a neural network, while the parameters of the preceding layers are represented by ϕ . For simplicity, we denote $\mathbf{w} = (w_1, \dots, w_n)$ as the collection of task-specific parameters. The objective function of meta-learning is given by

$$\begin{aligned} \min_{\phi} L_{\mathcal{D}}(\phi, \mathbf{w}^*) &= \frac{1}{n} \sum_{i=1}^n \ell_i(\phi) \\ \text{s.t. } \mathbf{w}^* &= \arg \min_{\mathbf{w}} \mathcal{L}_{\mathcal{S}}(\phi, \mathbf{w}) := \frac{1}{n} \sum_{i=1}^n \mathcal{L}_{\mathcal{S}_i}(\phi, w_i), \end{aligned} \quad (1)$$

where $\ell_i(\phi) = \frac{1}{|\mathcal{D}_i|} \sum_{\xi \in \mathcal{D}_i} \mathcal{L}(\phi, w_i^*; \xi)$ and $\mathcal{L}_{\mathcal{S}_i}(\phi, w_i) = \frac{1}{|\mathcal{S}_i|} \sum_{\xi \in \mathcal{S}_i} \mathcal{L}(\phi, w_i; \xi)$. This is a bi-level optimization problem (Bracken and McGill 1973; Ji, Yang, and Liang 2021), which can be solved in two stages. In the inner learning stage, the base learner of each task $\mathcal{T}_i$ searches the minimizer w_i^* of its loss over a support set $\mathcal{S}_i$. In the outer learning stage, the meta-learner evaluates the minimizers w_i^* on a held-out query set $\mathcal{D}_i$ and optimizes ϕ across all tasks.

As we mentioned in the Introduction, most existing meta-learning methods assume all tasks have equal influence on the meta-model. However, in real-world applications, tasks are often heterogeneous in terms of learning difficulty, data quality, and consistency. In the following experiments, we show the influence of heterogeneous tasks on meta-learning. **Influences of the Majority Tasks.** *Setting:* We train two meta-models, one using all tasks in each mini-batch of size 16 for updating meta-model parameters (traditional learning), and the other using only the top-8 tasks with the highest individual task losses (hard task mining). We evaluate both models by fast adaptation to all training tasks and testing them on the same test dataset. The distributions of task query losses and the test accuracy are presented in Figure 2 (a). *Analysis:* Using hard task mining during meta-training leads to lower task losses compared to using all tasks, indicating that easy tasks dominate training. Moreover, training with “hard” tasks improves the generalization of the meta-model, as demonstrated by the increased testing accuracy.

Sensitive to Noisy Tasks. *Setting:* We generate two training sets: clean and noisy, where 60% of the tasks contain samples with flipped labels. Then, we trained two meta-models on these two datasets separately and conducted fast adaptation on all clean tasks and noisy tasks respectively in the noisy training set. The distributions of training task losses together with test accuracy are shown in Figure 2 (b). *Analysis:* When trained with a noisy dataset, there is a significant overlap between the task loss distributions of noisy and clean tasks (yellow and green lines, respectively). However, when trained with a clean dataset, there is a clear separation between the two distributions, indicating that the meta-model trained with the noisy dataset cannot effectively distinguish between noisy and clean tasks. Additionally, training with noisy datasets results in higher clean task losses than clean training (blue line), while clean training results in higher noisy task losses than noisy training (red line). These illustrate the susceptibility of meta-learning to noisy tasks

and how it can harm the meta-model’s generalization ability, as indicated by the decline in test accuracy.

Vulnerability to the Outlier Tasks. *Setting:* Two categories with multiple classes from the Tiered-ImageNet (Ren et al. 2018a) dataset were randomly chosen as the target and outlier categories. Two meta-models were trained, the first one using 16 tasks randomly sampled from the target category as clean tasks in each iteration of the mini-batch, and the second one using 15 tasks from the target category (indexed 0 to 14) and 1 task from the outlier category (indexed as 15) in each iteration of the mini-batch. For the second one, we track the task with the highest loss in each iteration by recording its task index, and the frequency of each task index being the highest throughout the training process is plotted. The test accuracy of both models is also reported in Figure 2 (c). *Analysis:* The task from the outlier category frequently has the highest loss in each batch. Meta-learning is also vulnerable to outlier tasks, as evidenced by the lower test accuracy compared to the model trained without outliers.

4 Method

HeTRoM is based on a simple observation that hard, noisy, or outlier tasks are usually associated with large task-level losses during meta-training, while most tasks have relatively small losses throughout the training process. A similar observation can be made at the sample level in (Zhu et al. 2023; Jiang et al. 2018; Wang, Hu, and Hu 2020). As illustrated in Section 3, our findings also show that this phenomenon exists widely at the task level. Let $\mathbb{L} := \{\ell_1(\phi), \dots, \ell_n(\phi)\}$ be a set of all task losses on their corresponding query dataset. We also denote $\ell_{[i]}(\phi)$ as the i -th largest individual task loss after sorting the elements in set $\mathbb{L}$ (ties can be broken in any consistent way). With two integers k and m , $0 \leq m < k \leq n$, the HeTRoM loss is given by:

$$\begin{aligned} \min_{\phi} \mathcal{L}_{\mathcal{D}}(\phi, \mathbf{w}^*) &= \frac{1}{k-m} \sum_{i=m+1}^k \ell_{[i]}(\phi) \\ \text{s.t. } \mathbf{w}^* &= \arg \min_{\mathbf{w}} \mathcal{L}_{\mathcal{S}}(\phi, \mathbf{w}) := \frac{1}{n} \sum_{i=1}^n \mathcal{L}_{\mathcal{S}_i}(\phi, w_i). \end{aligned} \quad (2)$$

The HeTRoM loss is a generalization of ranked ranges in supervised learning (Hu et al. 2020). It is equivalent to a dynamic task selection method that filters imperfect tasks based on their individual task losses. Specifically, it selects tasks with small losses, which are more likely to be normal tasks, and drops the top- m tasks with large. It also excludes tasks with small losses (*i.e.*, top- k to top- n losses), which are the majority of tasks that are normal and potentially redundant. Conventional meta-learning is a special case of HeTRoM with $k = n$ and $m = 0$.

4.1 Algorithm

The ranking operation in HeTRoM can be eliminated using the following result.

Theorem 1 Denote $[a]_+ = \max\{0, a\}$ as the hinge func-

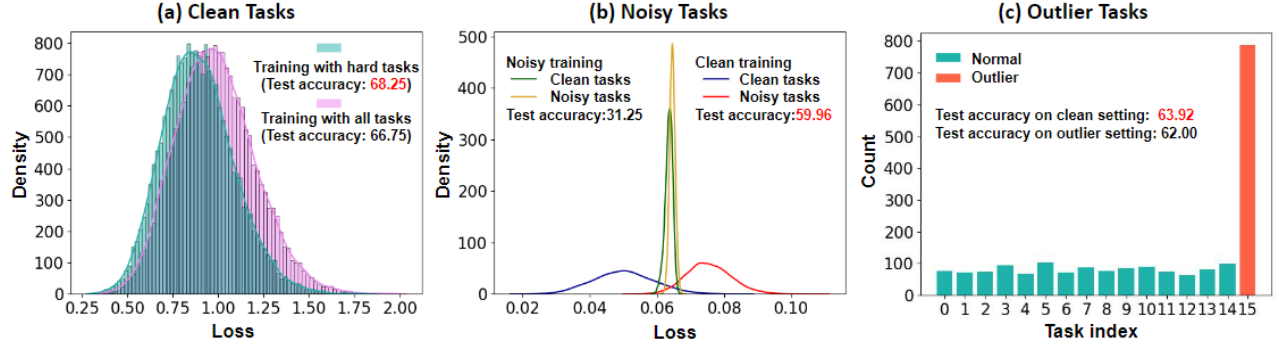


Figure 2: Behavior analysis of meta-learning. (a) The loss distributions of training tasks with models trained with all tasks and hard tasks, respectively. (b) The loss distributions of clean and noisy tasks with models trained with a clean dataset (“Clean training”) and a noisy dataset (“Noisy training”), respectively. (c) The frequency of each task index with the highest loss throughout the training process, where the outlier task is assigned index 15 in each iteration. “Test accuracy” (%) is obtained by testing on a separate clean test set.

tion. Equation (2) is equivalent to

$$\begin{aligned} \min_{\phi, \lambda} \max_{\hat{\lambda}} \hat{\mathcal{L}}_{\mathcal{D}}(\phi, \mathbf{w}^*) &= \frac{1}{k-m} \sum_{i=1}^n \hat{\mathcal{L}}(\phi, w_i^*, \lambda, \hat{\lambda}) \\ &:= \left[\ell_i(\phi) - \lambda \right]_+ - \left[\ell_i(\phi) - \hat{\lambda} \right]_+ + \frac{k}{n} \lambda - \frac{m}{n} \hat{\lambda} \\ \text{s.t. } \mathbf{w}^* &= \arg \min_{\mathbf{w}} \mathcal{L}_{\mathcal{S}}(\phi, \mathbf{w}) := \frac{1}{n} \sum_{i=1}^n \mathcal{L}_{\mathcal{S}_i}(\phi, w_i), \end{aligned} \quad (3)$$

If the optimal of ϕ and $\mathbf{w}^*$ are achieved, $(\ell_{[k]}(\phi), \ell_{[m]}(\phi))$ can be the optimum solutions of $(\lambda, \hat{\lambda})$.

The proof of Theorem 1 can be found in Appendix B. A bi-level optimization procedure (Ji, Yang, and Liang 2020) can be used to solve (3). Specifically, in each iteration j , we sample a batch of N tasks $\{\mathcal{T}_i\}_{i=1}^N$, and then each sampled task $\mathcal{T}_i$ conducts D steps of inner-loop updates as

$$w_{i,j}^t = w_{i,j}^{t-1} - \alpha \nabla_w \mathcal{L}_{\mathcal{S}_i}(\phi_j, w_{i,j}^{t-1}), \quad t = 1, \dots, D \quad (4)$$

where α is the inner-loop step size, $w_{i,j}^t$ is the task $\mathcal{T}_i$ ’s parameter at iteration j in step t , and $\nabla_w \mathcal{L}_{\mathcal{S}_i}$ is the gradient of $\mathcal{L}_{\mathcal{S}_i}$ with respect to (w.r.t.) w . Then, we need to update λ and $\hat{\lambda}$ together with the meta-model to keep track of the ranked range of task losses on the query set from top- m to top- k . To optimize the minimax problem in upper-level objective of equation (3), we use single gradient steps to update λ and $\hat{\lambda}$ simultaneously using their (sub)gradients given by

$$\begin{aligned} \partial_{\lambda} \hat{\mathcal{L}}(\phi_j, w_{i,j}^D, \lambda_j, \hat{\lambda}_j) &= \frac{k}{N} - \mathbb{I}_{[\ell_i(\phi_j, w_{i,j}^D) > \lambda_j]}, \\ \partial_{\hat{\lambda}} \hat{\mathcal{L}}(\phi_j, w_{i,j}^D, \lambda_j, \hat{\lambda}_j) &= \mathbb{I}_{[\ell_i(\phi_j, w_{i,j}^D) > \hat{\lambda}_j]} - \frac{m}{N} \end{aligned} \quad (5)$$

where $\ell_i(\phi_j, w_{i,j}^D)$ is the task $\mathcal{T}_i$ ’s loss on its query set using iteration j ’s meta-model parameters ϕ_j and task parameter $w_{i,j}^D$, and $\mathbb{I}_{[A]}$ is an indicator function, which returns 1 if the condition A is true. Finally, we adopt the stable iterative differentiation (ITD)-based approach (Grazzi et al. 2020) to efficiently approximate the gradient of $\hat{\mathcal{L}}_{\mathcal{D}}(\phi, \mathbf{w}^*)$ w.r.t. ϕ

Algorithm 1: Optimization procedure of HeTRoM

- 1: **Input:** Total iteration number J ; inner-loop iteration number D ; batch size N ; stepsizes α, β, γ , initial meta-parameter ϕ_0 ; initial task-specific parameter w_0 ; hyperparameters k, m ; $\lambda_0, \hat{\lambda}_0$.
 - 2: **for** $j = 0, 1, 2, \dots, J$ **do**
 - 3: Sample a batch of tasks $\{\mathcal{T}_i\}_{i=1}^N$
 - 4: **for** each task $\mathcal{T}_i$ in $\{\mathcal{T}_i\}_{i=1}^N$ **do**
 - 5: Initialize task-specific parameter as $w_{i,j}^0 = w_0$
 - 6: **for** $t = 1, 2, \dots, D$ **do**
 - 7: Update $w_{i,j}^t = w_{i,j}^{t-1} - \alpha \nabla_w \mathcal{L}_{\mathcal{S}_i}(\phi_j, w_{i,j}^{t-1})$
 - 8: **end for**
 - 9: **end for**
 - 10: $\begin{pmatrix} \lambda_{j+1} \\ \hat{\lambda}_{j+1} \end{pmatrix} \leftarrow \begin{pmatrix} \lambda_j \\ \hat{\lambda}_j \end{pmatrix} - \frac{\gamma}{k-m} \begin{pmatrix} \sum_{i=1}^N \partial_{\lambda} \hat{\mathcal{L}}(\phi_j, w_{i,j}^D, \lambda_j, \hat{\lambda}_j) \\ - \sum_{i=1}^N \partial_{\hat{\lambda}} \hat{\mathcal{L}}(\phi_j, w_{i,j}^D, \lambda_j, \hat{\lambda}_j) \end{pmatrix}$
based on equation (5)
 - 11: Update meta-parameter $\phi_{j+1} = \phi_j - \beta \text{Hg}(\phi_j)$ based on equation (6)
 - 12: **end for**
-

(which refers to as the hyper-gradient because of the implicit dependence of $\mathbf{w}^*$ on ϕ). In specific, we first evaluate the upper-level function over the inner-loop output $w_{i,j}^D$, and compute the hyper-gradient estimate as

$$\text{Hg}(\phi_j) := \frac{1}{k-m} \frac{\partial \sum_{i=1}^N \hat{\mathcal{L}}(\phi_j, w_{i,j}^D, \lambda_j, \hat{\lambda}_j)}{\partial \phi_j} \quad (6)$$

by conducting back-propagation over inner-loop trajectory w.r.t. ϕ_j . Following (Finn, Abbeel, and Levine 2017; Ji, Yang, and Liang 2021), the first-order hyper-gradient approximation $\frac{1}{k-m} \sum_{i=1}^N \partial_{\phi} \hat{\mathcal{L}}(\phi_j, w_{i,j}^D, \lambda_j, \hat{\lambda}_j)$ is employed to avoid the computation cost introduced by computing implicit derivative $\frac{\partial w_{i,j}^D}{\partial \phi_j}$ that contains second-order model information with a large model size. We show the entire optimization procedure of our method in Algorithm 1.

In practice, we follow the results obtained from Theorem

1 and initialize λ and $\hat{\lambda}$ through adaptation on a batch of randomly sampled tasks. Then, we sort the loss on the corresponding query sets and assign the k -th loss and m -th loss values to λ and $\hat{\lambda}$, respectively.

4.2 Determining the hyper-parameters

The values of hyper-parameters k and m are important to the overall performance of the model trained by Algorithm 1. We describe three practical measures to find their values.

Warm-up. Based on Theorem 1, we can determine how many task losses exceed the values of λ and $\hat{\lambda}$ by analyzing the values of k and m . Specifically, we use the conventional meta-learning approach to “warm up” the model based on equation (1) for a few iterations by training on the training data, which may contain noisy or outlier tasks. This procedure is similar to Algorithm 1. In particular, we have no λ and $\hat{\lambda}$ that need to be updated but we update meta-parameter ϕ using $\phi_{j+1} = \phi_j - \beta \text{Hg}(\phi_j)$, where $\text{Hg}(\phi_j) := \frac{1}{N} \frac{\partial \sum_{i=1}^N \ell_i(\phi_j, w_{i,j}^D)}{\partial \phi_j}$.

Identification. After the “warm-up”, we fast adapt the meta-model with parameters ϕ_{j+1} to all training tasks $\{\mathcal{T}_i\}_{i=1}^n$ using D steps of the inner-loop for each of them. Then we obtain the new trained task-specific parameters $\{\tilde{w}_i^D\}_{i=1}^n$. Next, we calculate all training task losses on their corresponding query sets so that we get $\{\ell_i(\phi_{j+1}, \tilde{w}_i^D)\}_{i=1}^n$. A histogram based on $\{\ell_i(\phi_{j+1}, \tilde{w}_i^D)\}_{i=1}^n$ is used to describe the training behaviors of all tasks. We can use different distributions to fit the histogram. However, in practice, the log-normal distribution can be a better fit. In addition, it is also popular to use log-normal distribution to fit training loss as (Holland 2023) in machine learning. Therefore, we take a log of all task losses as $\bar{\mathbb{L}} := \{\log \ell_i(\phi_{j+1}, \tilde{w}_i^D)\}_{i=1}^n$ and use their mean $\text{Mean}(\bar{\mathbb{L}})$ and standard deviation $\text{Std}(\bar{\mathbb{L}})$ as the descriptor for the statistic of task losses. In order to identify potential easy tasks and noisy or outlier tasks, we must define the thresholds that differentiate them from others. Specifically, we define the thresholds as:

$$\lambda^* = \exp(\text{Mean}(\bar{\mathbb{L}}) - \rho_1 \text{Std}(\bar{\mathbb{L}})), \quad (7)$$

$$\hat{\lambda}^* = \exp(\text{Mean}(\bar{\mathbb{L}}) + \rho_2 \text{Std}(\bar{\mathbb{L}})), \quad (8)$$

where ρ_1 and ρ_2 are hyper-parameters. In practice, we find that setting them within the range $[1, 3]$ yields good results.

Distillation. After we get the thresholds, we continue to train the meta-model obtained from the warm-up procedure. Thus, the meta-model parameters should be ϕ_{j+1} . Then, we use the thresholds to select tasks to update the meta-model parameters dynamically. In particular, tasks with their losses fall into $[\lambda^*, \hat{\lambda}^*]$ are regarded as useful tasks that benefit learning. The learning procedure is also similar to Algorithm 1. However, in the upper-level loop, we only need to use the following gradient to update meta-model parameters ϕ :

$$\widetilde{\text{Hg}}(\phi_{j+1}) := \frac{1}{|\mathcal{S}_\ell|} \frac{\partial \sum_{\ell_i \in \mathcal{S}_\ell} \ell_i}{\partial \phi_{j+1}} \quad (9)$$

where $\mathcal{S}_\ell = \mathbb{I}_{[\lambda^* \leq \ell_i(\phi_{j+1}, w_{i,j}^D) \leq \hat{\lambda}^*]}$ is the set containing task losses that fall between λ^* and $\hat{\lambda}^*$.

The corresponding Algorithm 2 is available in Appendix C. We call this approach statistic-guided HeTRoM (sg-HeTRoM). The significance of the thresholds can be more clearly understood as follows: λ^* and $\hat{\lambda}^*$ can be regarded as the optimal values of λ and $\hat{\lambda}$. In each iteration of the distillation process, we eliminate the impact of noisy or outlier tasks on the meta-parameter updates by using the upper bound $\hat{\lambda}^*$ of the task losses. Additionally, we enhance the meta-model’s generalization ability by potentially removing overly simple tasks based on the lower bound λ^* , which is akin to a hard task mining strategy.

5 Experiments

5.1 Experimental Settings

Datasets. We conduct our experiments in a few-shot classification setting, where two widely recognized datasets are used for few-shot classification: Mini-ImageNet (Vinyals et al. 2016), and Tiered-ImageNet (Ren et al. 2018a). Both are subsets of the ILSVRC-12 dataset (Russakovsky et al. 2015). Mini-ImageNet comprises 100 classes, with each class containing 600 images of 84×84 pixel dimensions. The dataset is divided into three subsets: 64 classes for training, 16 for validation, and 20 for testing. On the other hand, Tiered-ImageNet is a more substantial dataset, containing 779,165 images annotated across 608 classes. These classes are further grouped into 34 categories. This dataset is split at the category level, with 20 categories used for training, 6 for validation, and 8 for testing.

Baselines. We showcase the effectiveness of our approach by contrasting it with recent optimization-based meta-learning methods **aiming at enhancing the robustness to heterogeneous tasks** of meta-training, such as ATS (Yao et al. 2021) and Eigen-Reptile (Chen et al. 2022). Consequently, we do not directly compare our method with strategies that solely aim at boosting the meta-model’s performance (Baik et al. 2021, 2020; Collins, Mokhtari, and Shakkottai 2020). NestedMAML (Killamsetty et al. 2022) is not included in our comparison, as the original paper neither provides source code nor records performance on prevalent datasets like Mini-ImageNet and Tiered-ImageNet. Moreover, we examine the impact of our HeTRoM loss on the issues previously mentioned in the context of representation-based meta-learning methods. Importantly, our investigation is conducted without relying on training tracks, as is the case in (Antoniou, Edwards, and Storkey 2018). As a result, we benchmark our method against the MAML approach, which is widely acknowledged as a prevalent method in meta-learning. We do not compare with the most recent optimization-based meta-learning methods because our approach is versatile and can readily incorporate these methods. The detailed information on implementation can be found in Appendix D.

5.2 Results

Performance on Clean Tasks. We report the overall results (accuracy on the testing set) based on clean tasks in Table 1. Specifically, we use the HeTRoM loss and the statistically driven method to filter out non-informative and easy tasks

Dataset	Model	Method	Clean		Noisy (5-ways 5-shots)			Outlier (3-ways 5-shots)		
			5-ways 5-shots	5-ways 1-shot	Fixed	1-step	2-step	ratio=0	ratio=0.1	ratio=0.3
Mini-ImageNet	CNN4	MAML (Finn, Abbeel, and Levine 2017)	63.11 $\pm$ 0.91	48.70 $\pm$ 1.75	60.25	55.20	30.30	78.05	77.44	77.29
		ATS (Yao et al. 2021)	61.60 $\pm$ 0.69	50.75 $\pm$ 0.73	59.89	60.09	59.50	71.09	70.80	69.84
		Eigen-Reptile(Chen et al. 2022)	69.85 $\pm$ 0.85	53.25 $\pm$ 0.45	63.47	61.78	49.63	69.56	69.50	67.95
		HeTRoM (Ours)	70.45 $\pm$ 0.71	59.75 $\pm$ 1.28	68.75	67.30	61.45	81.25	78.96	80.63
	ResNet12	sg-HeTRoM (Ours)	70.08 $\pm$ 1.07	60.50 $\pm$ 2.03	67.58	68.00	67.67	79.86	79.17	78.12
		MAML (Finn, Abbeel, and Levine 2017)	69.54 $\pm$ 0.38	58.60 $\pm$ 0.42	64.50	58.25	44.75	79.30	77.78	77.29
Tiered-ImageNet	CNN4	HeTRoM (Ours)	75.35 $\pm$ 0.82	60.50 $\pm$ 2.32	71.25	64.50	54.33	87.35	86.21	80.77
		sg-HeTRoM (Ours)	71.87 $\pm$ 1.93	59.38 $\pm$ 1.84	70.13	67.25	59.75	86.18	85.50	80.01
	ResNet12	MAML (Finn, Abbeel, and Levine 2017)	66.25 $\pm$ 0.19	50.98 $\pm$ 0.26	34.50	31.25	31.50	63.92	62.83	62.00
		ATS(Yao et al. 2021)	60.44 $\pm$ 0.76	43.25 $\pm$ 0.84	58.72	57.12	49.06	48.69	48.64	48.02
		Eigen-Reptile (Chen et al. 2022)	57.36 $\pm$ 0.19	42.24 $\pm$ 0.18	56.05	53.37	52.16	59.01	57.17	55.41
		HeTRoM (Ours)	71.25 $\pm$ 0.57	58.50 $\pm$ 1.07	68.75	66.00	67.75	67.07	66.75	65.50
	ResNet12	sg-HeTRoM (Ours)	70.80 $\pm$ 1.74	58.75 $\pm$ 0.98	68.67	69.33	66.83	69.17	69.37	64.17
		MAML (Finn, Abbeel, and Levine 2017)	71.24 $\pm$ 0.43	58.58 $\pm$ 0.49	67.00	68.25	54.63	65.83	61.33	51.67
		HeTRoM (Ours)	72.25 $\pm$ 1.29	61.25 $\pm$ 0.82	69.17	64.88	67.75	66.25	65.00	58.75
		sg-HeTRoM (Ours)	71.25 $\pm$ 0.74	58.75 $\pm$ 1.64	69.33	70.25	68.25	67.08	64.17	59.16

Table 1: Test accuracy (%) on Mini-ImageNet and Tiered-ImageNet. For the clean setting, we report the average accuracy with 95% confidence intervals. The best results are shown in **Bold**. Gray indicates our methods outperform baseline methods in the same setting.

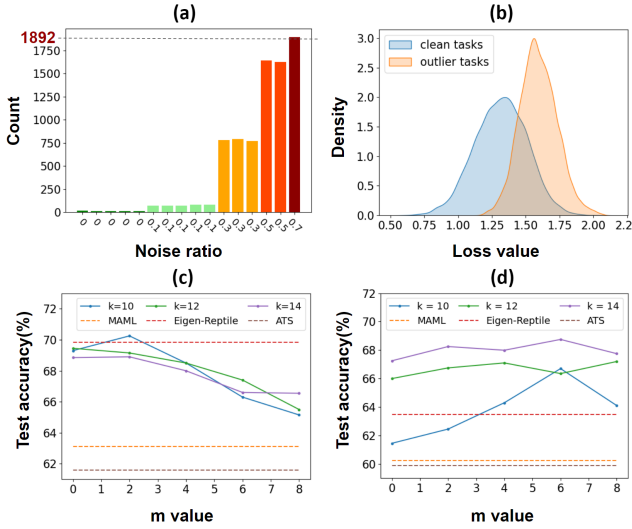


Figure 3: (a) Frequency of the noise ratio for each task in a mini-batch being excluded by $\hat{\lambda}$ during meta-training. (b) Loss distribution of clean and outlier tasks when conducting fast adaptation with a trained meta-model. (c) Test accuracy with varying k and m in clean task setting. (d) Test accuracy with varying k and m in fixed noisy task setting.

during training dynamically. We achieve a significant improvement in performance over the MAML method, with a [1.9%, 11.8%] margin. In addition, our method outperforms two robust meta-learning methods ATS and Eigen-Reptile on both datasets and two different few-shot settings (*i.e.*, 5-ways 5-shots and 5-ways 1-shot).

Performance on Noisy Tasks. Table 1 also shows the results of all methods on noisy task settings of 5-ways 5-shots classification. It is clear that all methods experience a significant decrease in performance in the presence of noise compared to the clean task scenario, but our methods still perform bet-

ter than the others. As we move from Mini-ImageNet to Tiered-ImageNet, the variety of tasks increases and MAML loses its generalization ability. This may be attributed to the fact that the number of easy tasks has increased significantly. On the other hand, ATS and Eigen-Reptile perform relatively well but still show a decreasing trend. In contrast, our methods consistently maintain stable or even better performance, indicating their robustness to the diversity of tasks in the presence of noisy tasks.

We then analyze in detail how HeTRoM works to enhance the robustness of MAML in a noisy task scenario under the *fixed* setting. We can keep track of the noise ratio for each task within a mini-batch since each task is indexed and fixed with one noisy ratio. This means that we can record the frequency of the noise ratio for each task excluded by $\hat{\lambda}$ (*i.e.*, identified as a noisy task) during meta-training. Figure 3 (a) displays this information. Our HeTRoM method can identify noisy tasks during meta-training, as demonstrated by the higher frequency of exclusion of tasks with a higher noise ratio. Clean tasks are rarely excluded, providing valuable information for training. Note that we do not require our method to exclude noisy tasks precisely in each iteration, which is not practical for an efficient method.

Performance on Outlier Tasks. Table 1 presents a comparison of our methods with other approaches on the outlier tasks setting of 3-ways 5-shots. We can observe that the performance of MAML shows a decreasing trend when increasing the ratio of outlier tasks. ATS and Eigen-Reptile fail to achieve competitive performance with MAML on both datasets, while our method demonstrates superior performance compared to MAML by a substantial margin. This can be attributed to the effective filtering of outlier tasks and too-easy tasks. To provide additional validation for the efficacy of removing outlier tasks during meta-training using our approach, we randomly sample 10,000 tasks from the Mini-ImageNet dataset (considered as clean) and the Tiered-ImageNet dataset (considered as outlier) respectively. A meta-model trained by our method performs fast adapta-

Architecture	Setting	Clean	Noisy		
			Fixed	1-step	2-step
CNN4	w/o HeTRoM	69.61 $\pm$ 0.38	67.36 $\pm$ 0.38	67.06 $\pm$ 0.39	63.35 $\pm$ 0.36
	w/HeTRoM	69.91 $\pm$ 0.37	67.65 $\pm$ 0.39	69.23 $\pm$ 0.38	65.23 $\pm$ 0.38
ResNet12	w/o HeTRoM	78.63 $\pm$ 0.46	76.76 $\pm$ 0.40	75.18 $\pm$ 0.40	72.40 $\pm$ 0.38
	w/HeTRoM	78.97 $\pm$ 0.34	78.61 $\pm$ 0.35	78.45 $\pm$ 0.35	75.95 $\pm$ 0.36

Table 2: Test accuracy (%) on Mini-ImageNet when incorporated with MetaOptNet (Lee et al. 2019). **Bold** indicates best results for each setting.

tion on all these tasks, and the distribution of task losses is depicted in Figure 3 (b). The results exhibit a clear trend wherein the task loss of outlier tasks tends to be larger than that of clean tasks, justifying the use of loss as a criterion for identifying outlier tasks.

Performance of sg-HeTRoM method. From Table 1, we can also observe the performance comparison between models trained with HeTRoM and sg-HeTRoM. The results indicate no significant performance gap between two methods. Moreover, both methods outperform all baselines, highlighting the effectiveness of leveraging statistical information to identify informative tasks and filter out noisy and outlier tasks. Furthermore, using a large model ResNet12 leads to improved performance in most settings. Nevertheless, our method still outperforms MAML.

Incorporating with SOTA Method. To ascertain the efficacy of our approach in conjunction with other state-of-the-art (SOTA) methodologies, we integrated our method with MetaOptNet(Lee et al. 2019) in both clean task and noisy task settings using Mini-ImageNet dataset. The results are shown in Table 2. It is evident from the results that the performance of MetaOptNet experiences a significant decline when confronted with noisy tasks. In contrast, our method demonstrates a mitigating effect on this performance degradation, particularly noteworthy in experiments with larger model (ResNet 12 in Table 2). More results on incorporating other SOTA methods can be found in Appendix E.

5.3 Discussion

k and m in Clean Setting. We analyze the effect on training with a variety of k and m in the clean task setting. As common settings, we conduct meta-training on the original Mini-ImageNet dataset with k chosen from $\{10, 12, 14\}$, and m chosen from $\{0, 2, 4, 6, 8\}$. The results are shown in Figure 3(c). The experimental results suggest two key findings: firstly, the best performance for each k is achieved with a small m value (e.g., $m = 2$), and performance diminishes with increasing k , indicating that easy tasks mainly influence meta-learning. Secondly, increasing m for a fixed k results in performance declines, underscoring the beneficial effect of incorporating hard tasks in training.

k and m in Noisy Setting. We study the impact of hyperparameters in the noisy task settings by conducting experiments in *fixed* noisy setting with a variety of k and m on Mini-ImageNet. The results are shown in Figure 3 (d). As m increases, there is a clear trend of improved performance, which can be attributed to the exclusion of more corrupted tasks from training, given the fact that the majority of tasks in a mini-batch are affected by varying levels of noise in

	$\rho_1 = 1$	$\rho_1 = 2$	$\rho_1 = 3$	$\rho_1 = \infty$
$\rho_2 = 1$	67.50	69.17	67.29	68.54
$\rho_2 = 2$	68.33	68.75	69.37	67.61
$\rho_2 = 3$	67.08	65.83	67.50	67.92
$\rho_2 = \infty$	67.08	66.77	67.67	67.37

Table 3: Effect of ρ_1 and ρ_2 on sg-HeTRoM in outlier task setting. $\rho_1 = \infty$ means no lower bound for task selection, and $\rho_2 = \infty$ means no upper bound for task selection.

fixed noise setting. Additionally, a larger value of k generally leads to higher performance, as it includes more clean tasks during training. Moreover, there is a clear range of k and m that our method outperforms the compared methods.

ρ_1 and ρ_2 in Outlier Setting. We conducted experiments on the Tiered-ImageNet dataset with the outlier setting (ratio=0.1) using sg-HeTRoM to study the impact of hyperparameters ρ_1 and ρ_2 in equation (7), which determine the thresholds for identifying easy tasks and outlier tasks. The results are presented in Table 3. We observe that sg-HeTRoM is not overly sensitive to ρ_1 and ρ_2 . If we set $\rho_2 = \infty$, changing the value of ρ_1 does not seem to affect the performance significantly. This could be because the outlier tasks significantly impact meta-learning more than the easy tasks. However, when we set $\rho_1 = \infty$, and ρ_2 varies, there is a visible fluctuation in the performance. This implies that ρ_2 is useful in mitigating the effect of outlier tasks.

6 Conclusion

Meta-learning provides a solution to handle few-shot scenarios when dealing with many tasks. Existing optimization-based meta-learning methods operate on the premise that all tasks are equally important. However, this assumption is often challenged in real-world scenarios where tasks are inherently heterogeneous, characterized by varying levels of difficulty, noisy training samples, or a significant divergence from the majority of tasks. In response to these challenges, we have developed a novel meta-learning approach that proficiently manages these heterogeneous tasks using rank-based task-level learning objectives. A key advantage of our method is its ability to handle outlier tasks effectively, and it prevents an over-representation of similar tasks in influencing the meta-learner. Our method also employs an efficient iterative optimization algorithm grounded in bi-level optimization. Our experimental evaluations demonstrate the flexibility of our approach, as it enables users to adapt to a variety of task settings and improves the overall performance of the meta-learner.

We plan to enhance our method in several ways in the future. Firstly, we aim to expand it to online meta-learning, where the model adjusts and learns in a real-time, sequential manner with the incoming data flow. Secondly, we intend to investigate its theoretical guarantees, including optimization error and generalization bound. Thirdly, we will test the effectiveness of our methods for other applications where meta-learning is helpful, such as object detection, speech recognition, and NLP.

References

- Alquier, P.; Pontil, M.; et al. 2017. Regret bounds for life-long learning. In *Artificial Intelligence and Statistics (AISTATS)*, 261–269.
- Antoniou, A.; Edwards, H.; and Storkey, A. 2018. How to train your MAML. *arXiv preprint arXiv:1810.09502*.
- Baik, S.; Choi, J.; Kim, H.; Cho, D.; Min, J.; and Lee, K. M. 2021. Meta-learning with task-adaptive loss function for few-shot learning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 9465–9474.
- Baik, S.; Choi, M.; Choi, J.; Kim, H.; and Lee, K. M. 2020. Meta-learning with adaptive hyperparameters. *Advances in neural information processing systems*, 33: 20755–20765.
- Balcan, M.-F.; Khodak, M.; and Talwalkar, A. 2019. Provable Guarantees for Gradient-Based Meta-Learning. In *International Conference on Machine Learning (ICML)*, 424–433.
- Bertinetto, L.; Henriques, J. F.; Torr, P.; and Vedaldi, A. 2018. Meta-learning with differentiable closed-form solvers. In *International Conference on Learning Representations (ICLR)*.
- Boudiaf, M.; Masud, Z.; Rony, J.; Dolz, J.; Piantanida, P.; and Ayed, I. 2020. Transductive information maximization for few-shot learning. *arXiv preprint arXiv:2008.11297*.
- Bracken, J.; and McGill, J. T. 1973. Mathematical programs with optimization problems in the constraints. *Operations research*, 21(1): 37–44.
- Chen, D.; Wu, L.; Tang, S.; Yun, X.; Long, B.; and Zhuang, Y. 2022. Robust Meta-learning with Sampling Noise and Label Noise via Eigen-Reptile. In *International Conference on Machine Learning*, 3662–3678. PMLR.
- Collins, L.; Mokhtari, A.; and Shakkottai, S. 2020. Task-robust model-agnostic meta-learning. *Advances in Neural Information Processing Systems*, 33: 18860–18871.
- Finn, C.; Abbeel, P.; and Levine, S. 2017. Model-agnostic meta-learning for fast adaptation of deep networks. In *International conference on machine learning*, 1126–1135. PMLR.
- Finn, C.; Rajeswaran, A.; Kakade, S.; and Levine, S. 2019. Online Meta-Learning. In *International Conference on Machine Learning (ICML)*, 1920–1930.
- Grant, E.; Finn, C.; Levine, S.; Darrell, T.; and Griffiths, T. 2018. Recasting gradient-based meta-learning as hierarchical bayes. In *International Conference on Learning Representations (ICLR)*.
- Grazzi, R.; Franceschi, L.; Pontil, M.; and Salzo, S. 2020. On the iteration complexity of hypergradient computation. In *International Conference on Machine Learning*, 3748–3758. PMLR.
- Holland, M. J. 2023. Flexible risk design using bi-directional dispersion. In *International Conference on Artificial Intelligence and Statistics*, 1586–1623. PMLR.
- Hu, S.; Ying, Y.; Lyu, S.; et al. 2020. Learning by minimizing the sum of ranked range. *Advances in Neural Information Processing Systems*, 33: 21013–21023.
- Huisman, M.; Van Rijn, J. N.; and Plaat, A. 2021. A survey of deep meta-learning. *Artificial Intelligence Review*, 54(6): 4483–4541.
- Ji, K.; Yang, J.; and Liang, Y. 2020. Bilevel Optimization: Nonasymptotic Analysis and Faster Algorithms. *arXiv preprint arXiv:2010.07962*.
- Ji, K.; Yang, J.; and Liang, Y. 2021. Bilevel optimization: Convergence analysis and enhanced design. In *International conference on machine learning*, 4882–4892. PMLR.
- Jiang, L.; Zhou, Z.; Leung, T.; Li, L.-J.; and Fei-Fei, L. 2018. Mentornet: Learning data-driven curriculum for very deep neural networks on corrupted labels. In *International conference on machine learning*, 2304–2313. PMLR.
- Killamsetty, K.; Li, C.; Zhao, C.; Chen, F.; and Iyer, R. 2022. A Nested Bi-level Optimization Framework for Robust Few Shot Learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, 7176–7184.
- Lee, K.; Maji, S.; Ravichandran, A.; and Soatto, S. 2019. Meta-learning with differentiable convex optimization. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Lee, K.-H.; He, X.; Zhang, L.; and Yang, L. 2018. Cleanet: Transfer learning for scalable image classifier training with label noise. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, 5447–5456.
- Liang, K. J.; Rangrej, S. B.; Petrovic, V.; and Hassner, T. 2022. Few-shot learning with noisy labels. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 9089–9098.
- Liu, C.; Wang, Z.; Sahoo, D.; Fang, Y.; Zhang, K.; and Hoi, S. C. 2020. Adaptive task sampling for meta-learning. In *Computer Vision—ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XVIII* 16, 752–769. Springer.
- Liu, X.; Lyu, Y.; Jing, L.; Zeng, T.; and Yu, J. 2023. Not All Tasks Are Equal: A Parameter-Efficient Task Reweighting Method for Few-Shot Learning. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, 421–437. Springer.
- Mallem, S.; Hasnat, A.; and Nakib, A. 2023. Efficient Meta label correction based on Meta Learning and bi-level optimization. *Engineering Applications of Artificial Intelligence*, 117: 105517.
- Nguyen, C.; Do, T.-T.; and Carneiro, G. 2023. Task Weighting in Meta-learning with Trajectory Optimisation. *arXiv preprint arXiv:2301.01400*.
- Raghu, A.; Raghu, M.; Bengio, S.; and Vinyals, O. 2020. Rapid learning or feature reuse? towards understanding the effectiveness of MAML. In *International Conference on Learning Representations (ICLR)*.
- Ren, M.; Triantafillou, E.; Ravi, S.; Snell, J.; Swersky, K.; Tenenbaum, J. B.; Larochelle, H.; and Zemel, R. S. 2018a. Meta-learning for semi-supervised few-shot classification. *arXiv preprint arXiv:1803.00676*.

- Ren, M.; Zeng, W.; Yang, B.; and Urtasun, R. 2018b. Learning to reweight examples for robust deep learning. In *International conference on machine learning*, 4334–4343. PMLR.
- Russakovsky, O.; Deng, J.; Su, H.; Krause, J.; Satheesh, S.; Ma, S.; Huang, Z.; Karpathy, A.; Khosla, A.; Bernstein, M.; et al. 2015. Imagenet large scale visual recognition challenge. *International journal of computer vision*, 115: 211–252.
- Sun, H.; Guo, C.; Wei, Q.; Han, Z.; and Yin, Y. 2022. Learning to rectify for robust learning with noisy labels. *Pattern Recognition*, 124: 108467.
- Vinyals, O.; Blundell, C.; Lillicrap, T.; Wierstra, D.; et al. 2016. Matching networks for one shot learning. *Advances in neural information processing systems*, 29.
- Wang, Z.; Hu, G.; and Hu, Q. 2020. Training noise-robust deep neural networks via meta-learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 4524–4533.
- Yao, H.; Wang, Y.; Wei, Y.; Zhao, P.; Mahdavi, M.; Lian, D.; and Finn, C. 2021. Meta-learning with an adaptive task scheduler. *Advances in Neural Information Processing Systems*, 34: 7497–7509.
- Zhou, F.; Wu, B.; and Li, Z. 2018. Deep meta-learning: Learning to learn in the concept space. *arXiv preprint arXiv:1802.03596*.
- Zhou, P.; Yuan, X.; Xu, H.; Yan, S.; and Feng, J. 2019. Efficient Meta Learning via Minibatch Proximal Update. In *Advances in Neural Information Processing Systems (NeurIPS)*, 1532–1542.
- Zhu, M.; Zhang, L.; Wang, L.; Li, D.; Zhang, J.; and Yi, Z. 2023. Robust co-teaching learning with consistency-based noisy label correction for medical image classification. *International Journal of Computer Assisted Radiology and Surgery*, 18(4): 675–683.

Appendix

A Settings for the Experiments in Section 3

Influence of Majority Tasks. In this setting, we use CNN4 as a backbone and train 2 meta-models on the Mini-ImageNet dataset with a mini-batch of size 16. During the meta-training phase, the two meta-models are updated iteratively using the entire mini-batch of tasks (traditional learning) and tasks with the top-8 highest losses (hard task mining) in each iteration. After 2000 iterations of training, we perform fast adaptation on all the training tasks and examine the distribution of task losses on the query set. Additionally, the test accuracy of both meta-models on the same test dataset is presented in Figure 2(a).

Sensitive to Noisy Tasks. In this setting, we utilize two distinct datasets: clean dataset and noisy dataset. The clean dataset consists of 20,000 tasks generated from the original Mini-ImageNet dataset, while the noisy dataset involves corrupting 60% of the tasks by randomly flipping all labels within each task. We train two separate meta-models on these datasets and subsequently perform fast adaptation on all tasks within the noisy dataset. Next, we separate the task losses of clean tasks (40% of all tasks) and noisy tasks (60% of all tasks) and show their respective distribution in Figure 2 (b). Note that the test accuracy shown in the figure corresponds to the performance evaluation conducted on the same clean test dataset.

Vulnerable to Outlier Task. As described in 5.1, the Tiered-ImageNet dataset is divided into categories, each containing a series of classes. In this setting, we choose two categories as target category and outlier category respectively. Two meta-models were trained, the first one using 16 tasks randomly sampled from the target category as clean tasks in each iteration of the mini-batch, and the second one using 15 tasks from the target category (indexed from 0 to 14) and 1 task from the outlier category (indexed as 15) in each iteration of the mini-batch. During training, we monitor the behavior of meta-training involving outlier tasks by recording the index of tasks with the highest task loss within the mini-batch in each iteration. Then we plot the frequency of each task index being the highest throughout the training process, which is shown in Figure 2 (c) together with the test accuracy of these two models on the same test dataset.

B Proof of Theorem 1

To prove Theorem 1, we first introduce the following lemma.

Lemma 1 ((Hu et al. 2020)) *For a set of task losses $\mathbb{L} = \{\ell_1(\phi), \dots, \ell_n(\phi)\}$, and $\ell_{[i]}$ denotes the i -th largest loss after sorting the elements in $\mathbb{L}$, we have*

$$\sum_{i=1}^k \ell_{[i]}(\phi) = \min_{\lambda \in \mathbb{R}} \left\{ k\lambda + \sum_{i=1}^n [\ell_i(\phi) - \lambda]_+ \right\}.$$

Furthermore, $\ell_{[k]}(\phi) \in \arg \min_{\lambda \in \mathbb{R}} \{k\lambda + \sum_{i=1}^n [\ell_i(\phi) - \lambda]_+\}$.

Proof 1 We know $\sum_{i=1}^k \ell_{[i]}(\phi)$ is the solution of

$$\max_{\mathbf{p}} \mathbf{p}^\top \mathbb{L}, \quad \text{s.t. } \mathbf{p}^\top \mathbf{1} = k, \mathbf{0} \leq \mathbf{p} \leq \mathbf{1}.$$

We apply Lagrangian to this equation and get

$$L = -\mathbf{p}^\top \mathbb{L} - \mathbf{v}^\top \mathbf{p} + \mathbf{u}^\top (\mathbf{p} - \mathbf{1}) + \lambda (\mathbf{p}^\top \mathbf{1} - k),$$

where $\mathbf{u} \geq \mathbf{0}$, $\mathbf{v} \geq \mathbf{0}$, $\lambda \in \mathbb{R}$ are Lagrangian multipliers. Taking its derivative w.r.t $\mathbf{p}$ and set it to 0, we have $\mathbf{u} - \mathbb{L} + \lambda \mathbf{1}$. Substituting it back to Lagrangian, we have

$$\min_{\mathbf{u}, \lambda} \mathbf{u}^\top \mathbf{1} + k\lambda, \text{ s.t. } \mathbf{u} \geq \mathbf{0}, \mathbf{u} + \lambda \mathbf{1} - \mathbb{L} \geq \mathbf{0}.$$

This means

$$\sum_{i=1}^k \ell_{[i]}(\phi) = \min_{\lambda} \left\{ k\lambda + \sum_{i=1}^n [\ell_i(\phi) - \lambda]_+ \right\}. \quad (\text{B.1})$$

Furthermore, we can see that $\lambda = \ell_{[k]}(\phi)$ is always one optimal solution for Eq.B.1. So

$$\ell_{[k]}(\phi) \in \arg \min_{\lambda} \left\{ k\lambda + \sum_{i=1}^n [\ell_i(\phi) - \lambda]_+ \right\}.$$

Theorem B.1 (Theorem 1 restated) Denote $[a]_+ = \max\{0, a\}$ as the hinge function. Equation (2) is equivalent to

$$\begin{aligned} \min_{\phi, \lambda} \max_{\hat{\lambda}} \hat{\mathcal{L}}_{\mathcal{D}}(\phi, \mathbf{w}^*) &= \frac{1}{k-m} \sum_{i=1}^n \hat{\mathcal{L}}(\phi, w_i^*, \lambda, \hat{\lambda}) \\ &:= \left[[\ell_i(\phi) - \lambda]_+ - [\ell_i(\phi) - \hat{\lambda}]_+ + \frac{k}{n} \lambda - \frac{m}{n} \hat{\lambda} \right] \\ \text{s.t. } \mathbf{w}^* &= \arg \min_{\mathbf{w}} \mathcal{L}_{\mathcal{S}}(\phi, \mathbf{w}) := \frac{1}{n} \sum_{i=1}^n \mathcal{L}_{\mathcal{S}_i}(\phi, w_i), \end{aligned} \quad (\text{B.2})$$

If the optimal of ϕ and $\mathbf{w}^*$ are achieved, $(\ell_{[k]}(\phi), \ell_{[m]}(\phi))$ can be the optimum solutions of $(\lambda, \hat{\lambda})$.

Proof 2 From Lemma 1 and upper-level formulation in (2), we have

$$\begin{aligned} \min_{\phi} \mathcal{L}_{\mathcal{D}}(\phi, \mathbf{w}^*) &= \frac{1}{k-m} \min_{\phi} \sum_{i=m+1}^k \ell_{[i]}(\phi) \\ &= \frac{1}{k-m} \min_{\phi} \left[\sum_{i=1}^k \ell_{[i]}(\phi) - \sum_{i=1}^m \ell_{[i]}(\phi) \right] \\ &= \frac{1}{k-m} \min_{\phi} \left[\min_{\lambda \in \mathbb{R}} \left\{ k\lambda + \sum_{i=1}^n [\ell_i(\phi) - \lambda]_+ \right\} \right. \\ &\quad \left. - \min_{\hat{\lambda} \in \mathbb{R}} \left\{ m\hat{\lambda} + \sum_{i=1}^m [\ell_i(\phi) - \hat{\lambda}]_+ \right\} \right] \\ &= \frac{1}{k-m} \min_{\phi, \lambda} \max_{\hat{\lambda}} \sum_{i=1}^n \left[[\ell_i(\phi) - \lambda]_+ - [\ell_i(\phi) - \hat{\lambda}]_+ \right. \\ &\quad \left. + \frac{k}{n} \lambda - \frac{m}{n} \hat{\lambda} \right]. \end{aligned}$$

According to Lemma 1, if the optimal parameter ϕ^* is achieved, we have $\lambda = \ell_{[k]}(\phi^*)$ and $\hat{\lambda} = \ell_{[m]}(\phi^*)$. Since $k > m$, it is obvious that $\hat{\lambda} > \lambda$.

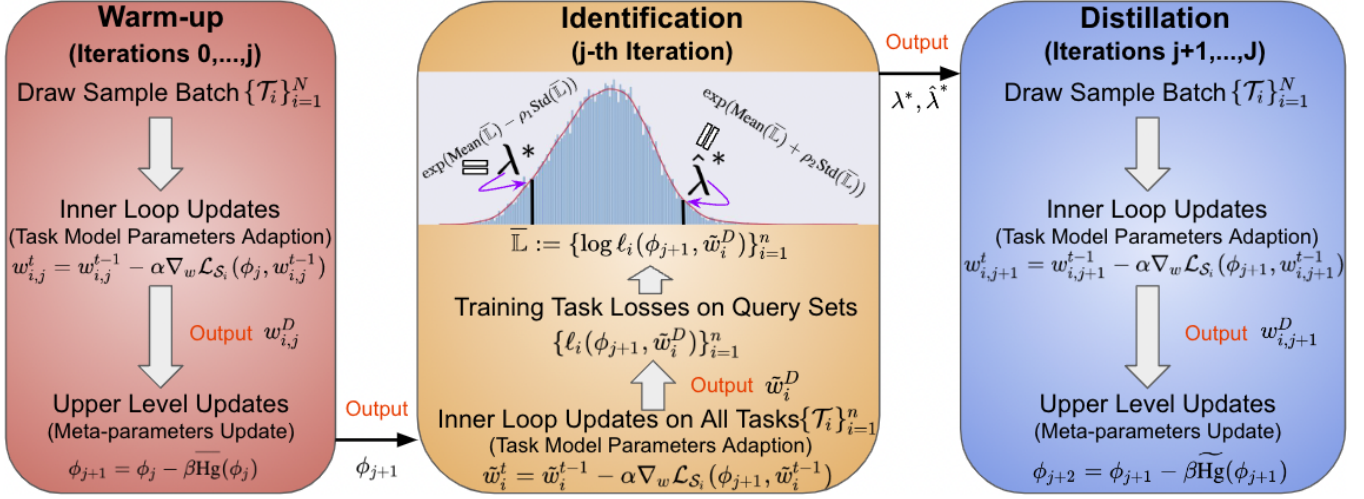


Figure B.1: Illustration of statistic-guided learning for solving HeTRoM.

C The Algorithm of sg-HeTRoM

In this section we provide comprehensive illustration of sg-HeTRoM method mentioned in 4.2, as in Figure B.1 and Algorithm 2.

D Implementation Details

Network Architecture. In this study, we employ two distinct network architectures, namely 4-layer CNN (CNN4) and ResNet12. CNN4 comprises four convolutional layers with a feature dimension of 64, each followed by a ReLU activation layer and max pooling. On the other hand, ResNet12 consists of four residual blocks, each containing three convolutional layers, along with batch normalization and ReLU activation.

Training Parameters. In the experimental section, we employ different hyperparameters for different network architectures. For both architectures, we fix the total number of iterations to $J = 2000$, and utilize Adam optimizer with a batch size of 16 throughout the training process. For CNN4, we set the inner-loop iteration number $D = 10$, and learning rates are configured as $\alpha = 0.05$, $\beta = \gamma = 0.002$. On the other hand, the inner-loop iteration number of ResNet12 $D = 3$, and the learning rates are $\alpha = 0.5$, $\beta = \gamma = 0.003$.

Implementation Details. We conduct our experiments in a few-shot classification setting. Two types of neural network backbones are used for meta-training: a 4-layer CNN (CNN4) and ResNet12. Our proposed method is trained using a mini-batch ADAM algorithm with a batch size of 16. The total iterations J are set to 2000, with inner-loop iterations D at 10. The initial learning rates are $\alpha = 0.05$, $\beta = \gamma = 0.002$. All our experiments are conducted using the PyTorch platform with two NVIDIA RTX A6000 GPUs. We construct our loss function based on the cross-entropy loss for the tasks. In line with (Hu et al. 2020), we conduct a grid search to select the values of k and m . We select ρ_1 and ρ_2 from the set $\{1, 2, 3\}$. The results we present, including a 95% confidence interval, are based on five random

runs. Unless stated otherwise, all experiments are conducted based on Algorithm 1. The detail of generating heterogeneous tasks can be found in Appendix D.

Learning Rate Strategy for Training ResNet12 Model.

Throughout the meta-training process of the ResNet12 model, the initial individual task losses are high but undergo a significant decrease as training progresses. In order to ensure that our thresholds, λ and $\hat{\lambda}$, are able to adapt to the changing scale of task losses, we introduce a scaling factor for the learning rate γ , denoted as δ . This scaling factor starts with a large value at the beginning of meta-training and linearly decreases over time, ultimately reaching 0 at the end of training. This procedure is denoted as $\gamma_i = i\delta\gamma_0/J$, where γ_i denotes the learning rate of λ and $\hat{\lambda}$ at i -th iteration, and γ_0 is the base learning rate. In our experiments, we set $\gamma_0 = \beta = 0.003$ and the scaling factor $\delta = 1000$.

The Generation of Clean Tasks. We designate the original Mini-ImageNet and Tiered-Imagenet datasets as clean datasets, from which we extract tasks for conducting experiments in the clean task setting. Our experiments focus on two few-shot classification tasks: 5-ways 5-shots, and 5-ways 1-shot. In both tasks, the “way” number refers to the number of classes in the task and the “shot” number indicates the number of examples sampled from each class.

The Generation of Noisy Tasks. we consider three generation settings for noisy tasks: fixed, 1-step random, and 2-step random. (a) In the *fixed* setting, we assign a specific noise probability for each of the tasks in a batch and then change the label of each sample to another randomly chosen label in each task according to the preset probability. For example, consider a batch of 16 tasks, where the first five tasks have no noise, the next five tasks have 10% noise each, the following three tasks have 30% noise each, the 14th and 15th tasks have 50% noise each, and the 16th task has 70% noise in each. This setting can help us track the noise in each batch to show the process of eliminating noisy tasks from training. (b) In *1-step random* setting, for each task, we randomly choose its noise probability from

Algorithm 2: Optimizing procedure of sg-HeTRoM

```

1: Input: Warmup iteration number  $P$ ; total iteration number  $J$ ; inner-loop iteration number  $D$ ; batch size  $N$ ; stepsize  $\alpha, \beta, \gamma$ ,
   initial meta-parameter  $\phi_0$ ; initial task-specific parameter  $w_0$ ; warmup flag  $flag = 1$ ; hyperparameters  $\rho_1, \rho_2$ .
2: for  $j = 0, 1, 2, \dots, J$  do
3:   Sample a batch of tasks  $\{\mathcal{T}_i\}_{i=1}^N$ 
4:   for each task  $\mathcal{T}_i$  in  $\{\mathcal{T}_i\}_{i=1}^N$  do
5:     Initialize task-specific parameter as  $w_{i,j}^0 = w_0$ 
6:     for  $t = 1, 2, \dots, D$  do
7:       Update  $w_{i,j}^t = w_{i,j}^{t-1} - \alpha \nabla_w \mathcal{L}_{\mathcal{S}_i}(\phi_j, w_{i,j}^{t-1})$ 
8:     end for
9:   end for
10:  if  $flag == 1$  then
11:    Calculate hypergradient of  $L_{\mathcal{D}}(\phi, \mathbf{w}^*)$  w.r.t.  $\phi$ :  $\overline{\text{Hg}}(\phi_j) = \frac{1}{N} \frac{\partial \sum_{i=1}^N \ell_i(\phi_j, w_{i,j}^D)}{\partial \phi_j}$ 
12:    Update meta-parameter  $\phi_{j+1} = \phi_j - \beta \overline{\text{Hg}}(\phi_j)$ 
13:    if  $j == P$  then
14:      for each task  $\mathcal{T}_i$  in the training set of all tasks  $\{\mathcal{T}_i\}_{i=1}^n$  do
15:        Initialize task-specific parameter as  $\tilde{w}_i^0 = w_0$ 
16:        for  $t = 1, 2, \dots, D$  do
17:          Update  $\tilde{w}_i^t = \tilde{w}_i^{t-1} - \alpha \nabla_w \mathcal{L}_{\mathcal{S}_i}(\phi_{j+1}, \tilde{w}_i^{t-1})$ 
18:        end for
19:      end for
20:      Calculate log of all training task losses on their query set:  $\overline{\mathbb{L}} = \{\log \ell_i(\phi_{j+1}, \tilde{w}_i^D)\}_{i=1}^n$ 
21:      Calculate+.  $\begin{pmatrix} \lambda^* \\ \hat{\lambda}^* \end{pmatrix} \leftarrow \begin{pmatrix} \exp(\text{Mean}(\overline{\mathbb{L}}) - \rho_1 \text{Std}(\overline{\mathbb{L}})) \\ \exp(\text{Mean}(\overline{\mathbb{L}}) + \rho_2 \text{Std}(\overline{\mathbb{L}})) \end{pmatrix}$ 
22:       $flag \leftarrow 0$ 
23:    end if
24:  else
25:    Update meta-parameter  $\phi_{j+1} = \phi_j - \beta \widetilde{\text{Hg}}(\phi_j)$  based on Eq. (9)
26:  end if
27: end for

```

{0%, 10%, 20%, 30%, 40%}. (c) In *2-step random*, following (Yao et al. 2021), we first randomly select 60% of tasks from all training tasks and set their noise probability to 80%.

The Generation of Outlier Tasks. (a) For the Tiered-ImageNet dataset, we consider the preset categories in Tiered-ImageNet as different domains and randomly select two categories: one is designated as the target category, and the other as the outlier category. We split all classes in each category into separate training and test parts with a 7:3 ratio. During the training phase, we randomly select a portion (*i.e.*, 10%, 30%) of tasks as outlier tasks, whose samples are extracted from the training part of the outlier category. Samples from other tasks are taken from the training part of the target category. During the test phase, all tasks are sampled from the test part of the target category. (b) For Mini-ImageNet dataset, we cross-reference the classes in Mini-ImageNet with the classes in Tiered-ImageNet to determine the category to which each class in Mini-ImageNet belongs. This allows us to identify the categories included in Mini-ImageNet, and the remaining categories in Tiered-ImageNet are considered outlier categories. In training phase, we sample a portion (*i.e.*, 10%, 30%) of tasks from outlier categories as outlier tasks, while the remaining tasks are sampled from Mini-ImageNet. During test phase, we exclusively evaluate the model’s performance on Mini-ImageNet.

Methods	MAML	ATS	Eigen-Reptile	Ours
Time	1.22s	2.68s	3.59s	1.36s

Table E.1: Average iteration time of our method and baseline methods.

E Additional Experimental Results

Time complexity analysis. The time complexity of our method is comparable to MAML (Finn, Abbeel, and Levine 2017), requiring only the maintenance of two extra scalar variables thus ensuring efficiency relative to other SOTA methods, which typically require additional computations (*e.g.*, weight calculation in (Yao et al. 2021)). In Table E.1 we display the average iteration time of our method and baseline methods, showing that our method can achieve better efficiency.

Meta-learning with Extreme Few Tasks. In real-world scenarios, users may face challenges accessing a substantial number of tasks for training the meta-model. We conducted experiments under the constraint of a limited task set (specifically, 100 tasks in this setting) during training, and the outcomes are presented in Table E.2. The results reveal a noteworthy performance decline for MAML compared to the experiments detailed in Table 1, rendering it impractical

Dataset	Method	Clean		Noisy (5-ways 5-shots)			Outlier (3-ways 5-shots)		
		5w5s	5w1s	Fixed	1-step	2-step	ratio=0	ratio=0.1	ratio=0.3
Mini-ImageNet	MAML	48.72	42.25	41.00	41.85	38.95	65.83	65.33	64.58
	HeTRoM(Ours)	59.90	46.25	59.45	59.15	58.25	75.42	72.50	71.25
	sg-HeTRoM(Ours)	58.94	46.50	57.60	57.25	56.75	79.41	78.75	77.08
Tiered-ImageNet	MAML	56.10	46.25	52.85	52.60	47.85	59.41	58.50	57.40
	HeTRoM(Ours)	62.75	49.50	62.10	61.65	61.65	59.75	59.50	59.42
	sg-HeTRoM(Ours)	61.75	47.25	60.89	61.25	60.10	64.79	62.58	62.17

Table E.2: Test accuracy (%) on Mini-ImageNet and Tiered-ImageNet using CNN4 architecture with 100 training tasks. The best results are shown in **Bold** for each setting. Gray indicates our methods outperform baseline methods in the same setting.

Model	Method	Clean		60% noisy samples	
		5w5s	5w1s	5w5s	5w1s
ResNet18	TIM w/o HeTRoM	79.11 $\pm$ 0.14	59.53 $\pm$ 0.21	73.56 $\pm$ 0.21	57.21 $\pm$ 0.21
	TIM w HeTRoM	80.16 $\pm$ 0.14	61.03 $\pm$ 0.20	74.48 $\pm$ 0.16	57.20 $\pm$ 0.20
WideResNet28-10	TIM w/o HeTRoM	82.81 $\pm$ 0.13	62.59 $\pm$ 0.20	74.02 $\pm$ 0.16	57.37 $\pm$ 0.21
	TIM w HeTRoM	82.68 $\pm$ 0.14	63.35 $\pm$ 0.20	74.28 $\pm$ 0.16	57.49 $\pm$ 0.21

Table E.3: Test accuracy on MiniImageNet compared with TIM.

in such constrained settings. In contrast, our method demonstrates appropriate performance, with particularly noteworthy efficacy in noisy settings.

Incorporating with SOTA few-shot learning method.

In addition to SOTA meta-learning techniques, our sg-HeTRoM method is readily adaptable to other few-shot learning approaches with various training paradigms. We apply our sg-HeTRoM method at the sample level to integrate with TIM (Boudiaf et al. 2020), with the results presented in Table E.3. To be specific, we apply the same model settings as in (Boudiaf et al. 2020), and use the clean and 2-step random noisy setting as in Section 5.1. In implementing our sg-HeTRoM approach at the sample level, we continue to adhere to the Warm-up, Identification, and Distillation framework specified in Section 4.2, with the adaptation that the Identification phase is executed at the sample level. The results demonstrate that incorporating our method allows TIM to attain comparable or even superior performance across different model architectures in both clean and noisy environments, underscoring the effectiveness of our approach when combined with SOTA methods.